

Java PrintStream Class - Explanation & Examples

The `PrintStream` class in Java is part of the `java.io` package and provides methods to print output to different destinations like the console, files, or output streams.

Key Features of PrintStream

1. **Prints data to an output stream** (e.g., console, file).
2. **Supports primitive types** (`int`, `double`, `char`, etc.) and `Object`.
3. **No need for explicit flushing**—it automatically flushes the output on `println()`.
4. **Does not throw IOException**—instead, it sets an internal error flag.

Commonly Used PrintStream Methods

Method	Description
<code>print()</code>	Prints text without a newline
<code>println()</code>	Prints text and moves to the next line
<code>printf()</code>	Formats and prints output
<code>write(int b)</code>	Writes a single byte
<code>flush()</code>	Flushes the stream manually
<code>close()</code>	Closes the stream

1. PrintStream with System.out (Default Usage)

By default, Java uses `System.out`, which is a `PrintStream`, to print output to the console.

Java code:

```
public class PrintStreamExample {
    public static void main(String[] args) {
        PrintStream ps = System.out; // Default console output
        ps.println("Hello, PrintStream!");
        ps.print("This is printed on the same line.");
        ps.printf(" Value of Pi: %.2f%n", Math.PI);
    }
}
```

Output:

```
pgsql
CopyEdit
Hello, PrintStream!
This is printed on the same line. Value of Pi: 3.14
```

2. Writing Output to a File using PrintStream

You can redirect output to a file instead of the console.

```

Java code:
import java.io.*;

public class PrintStreamToFile {
    public static void main(String[] args) {
        try {
            PrintStream ps = new PrintStream(new File("output.txt"));
            ps.println("This is written to a file.");
            ps.println(100); // Printing an integer
            ps.println(99.99); // Printing a double
            ps.close();
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

Content of `output.txt` after execution:

```

This is written to a file.
100
99.99

```

3. Redirecting `System.out` to a File

You can redirect `System.out` so that all console output goes to a file.

```

Java code:
import java.io.*;

public class RedirectSystemOut {
    public static void main(String[] args) {
        try {
            PrintStream fileOut = new PrintStream("log.txt");
            System.setOut(fileOut); // Redirect console output to a file
            System.out.println("This will be written to log.txt instead of
the console.");
        } catch (FileNotFoundException e) {
            e.printStackTrace();
        }
    }
}

```

Content of `log.txt`:

```

This will be written to log.txt instead of the console.

```

4. `PrintStream` with `ByteArrayOutputStream` (Capturing Output)

You can capture `System.out` output in memory using `ByteArrayOutputStream`.

```

java

```

```
java code:
import java.io.*;

public class PrintStreamMemoryExample {
    public static void main(String[] args) {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        PrintStream ps = new PrintStream(baos);
        System.setOut(ps); // Redirect output to memory

        System.out.println("Captured Output!");
        System.out.println(12345);

        String capturedOutput = baos.toString();
        System.setOut(new PrintStream(new
        FileOutputStream(FileDescriptor.out))); // Reset to console

        System.out.println("Captured Text:");
        System.out.println(capturedOutput);
    }
}
```

Output on Console:

```
Captured Text:
Captured Output!
12345
```

5. Using `printf()` for Formatted Output

You can use `printf()` to format output similar to `printf` in C.

```
Java code:
public class PrintfExample {
    public static void main(String[] args) {
        PrintStream ps = System.out;
        ps.printf("Name: %s, Age: %d, Height: %.2f meters\n", "Alice", 25,
1.75);
    }
}
```

Output:

```
Name: Alice, Age: 25, Height: 1.75 meters
```

Key Takeaways

1. `PrintStream` is used for printing data to different output destinations.
2. `System.out` is an instance of `PrintStream`.
3. It **does not throw** `IOException` but can set an error flag.
4. It supports automatic flushing when `println()` is used.
5. You can redirect output to files or memory using `setOut()`.